Algorytm Min-Max

Oskar Sałata

Kwiecień 2026

Wprowadzenie do sztucznej inteligencji,
Politechnika Warszawska

1 Wprowadzenie

Trzecie zadanie z przedmiotu *Wprowadzenie do sztucznej inteligencji* polegało na zaimplementowaniu algorytmu Min-Max z przecinaniem Alfa-Beta oraz przebadaniu go dla różnych przypadków i parametrów. Algorytm ten miał za zadanie rywalizować z graczem w grze kółko i krzyżyk, która była już przygotowana przez prowadzących.

Algorytm działa rekurencyjnie na bazie drzewa gry. Na początku algorytm przyjmuje stan gry i jeżeli jest on końcowy, to przypisuje mu wartość w zależności od tego czy wygrywa któryś z graczy bądź jest remis. Skuteczność algorytmu zależy bezpośrednio od ustawionego limitu głębokości przeszukiwania.

2 Algorytm Min-Max i gracz losowy

2.1 Plansza 3x3

Maksymalna głębokość algorytmu jest standardowo ustawiona na 9, tak aby był w stanie przeanalizować każdy scenariusz. Rozgrywka pomiędzy Min-Maxem a graczem wykorzystującym losowe ruchy prawie zawsze kończy się porażką losowego gracza. Jest to spowodowane tym, że Min-Max może w najgorszym scenariuszu doprowadzić do remisu, ale zawsze będzie grał optymalnie. Gracz losowy może co najwyżej przez przypadek wybierać ruchy, które spowodują, że rozgrywka zakończy się remisem.

Tabela 1: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 9

Wygrana gracza Min-Max	Remis	Wygrana gracza losowego
20	0	0

Tabela 2: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 4

Wygrana gracza Min-Max	Remis	Wygrana gracza losowego
19	1	0

Tabela 3: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 1

Wygrana gracza Min-Max	Remis	Wygrana gracza losowego
17	0	3

Dla głębokości wynoszącej 1 algorytm działa tak, że wybiera pozycję, która tylko w kolejnej turze zwraca najlepszy rezultat, z tego powodu, jeżeli losowy gracz wybierze pozycje, która blokuje zwycięstwo gracza Min-Max, a do tego wybierze pola, które później ułożą się w wygrywającą sekwencje to gracz losowy wygra. Ten scenariusz polega na czystym przypadku.

Rysunek 1: Wygrana gracza losowego dla głębokości równej 1

```
INFO:root:Moving: P[x][RandomPlayer]
INFO:root:Player selected 7
INFO:root:Move done.

Turn 8 moving: o, player P[o][MinMaxPlayer]
 ---
|oox|
|oo |
|xxx|
 ---

RESULT:x:P[x][RandomPlayer]
Time played: {'o': 0.11420249938964844, 'x': 0.008821487426757812}
```

2.2 Plansza 4x4

W przypadku zwiększenia rozmiaru planszy wyniki się zmieniają. Po pierwsze jest mniejsze prawdopodobieństwo wygranej gracza losowego, ale z drugiej strony jest większa szansa na scenariusz, w którym wynikiem końcowym jest remis.

Tabela 4: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 1

Wygrana gracza Min-Max	Remis	Wygrana gracza losowego
11	6	4

Tabela 5: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 6

Wygrana gracza Min-Max	Remis	Wygrana gracza losowego
12	8	0

Rysunek 2: Remis dla głębokości równej 1

```
INFO:root:Moving: P[x][RandomPlayer]
INFO:root:Player selected 15
INFO:root:Move done.

Turn 16 moving: o, player P[o][MinMaxPlayer]
 ----
|ooxo|
|ooxx|
|xoxo|
|xxox|
 ----

RESULT:DRAW
Time played: {'o': 648.9453315734863, 'x': 0.028371810913085938}
```

Interesującym zjawiskiem jest długość rozgrywki. Plansza 4x4 powoduje, że całkowity czas poświęcony na decyzje gracza Min-Max jest wielokrotnie większy w porównaniu do losowego gracza.

3 Algorytm Min-Max i gracz

3.1 Plansza 3x3

Tabela 6: Wyniki poszczególnych graczy dla 20 rozgrywek przy głębokości równej 9

Wygrana gracza Min-Max	Remis	Wygrana człowieka
5	15	0

Przedstawione wyniki potwierdzają, że algorytm przeszukujący pełne drzewo gry jest przeciwnikiem niepokonanym, eliminującym jakiekolwiek ryzyko wyboru przegrywającego stanu. Z drugiej strony, gracz ludzki zazwyczaj

potrafi poprawnie analizować niewielką planszę 3x3 i odpowiednio reagować na zagrożenia. Starcie bezbłędnie broniącego się algorytmu z racjonalnie grającym człowiekiem prowadzi w zdecydowanej większości przypadków do wyniku nierozstrzygniętego.

Rysunek 3: Widły wykonane przeciwko graczowi MinMax

```
Turn 7 moving: x, player P[x][MinMaxPlayer]
 ---
|oxo|
|xox|
|o  |
 ---

RESULT:o:P[o][HumanPlayer]
Time played: {'o': 23361.73915863037, 'x': 1.367807388305664}
```

Dla głębokości przeszukiwania równej 2, algorytm Min-Max ulega jedynie złożonym pułapkom w postaci podwójnego ataku. Agent ten prawidłowo wybiera ruchy chroniące go przed przegraną w najbliższej turze, jednak w obliczu wideł każda opcja ratunku jest przez niego oceniana jako przegrywająca. Algorytm zaprzestaje wówczas aktywnej obrony zagrożonych linii i wybiera dowolne wolne pole, sprawiając wrażenie poddania gry.

3.2 Plansza 4x4

W przypadku planszy 4x4 kluczowym wyzwaniem staje się optymalny dobór głębokości przeszukiwania. Ustawienie zbyt wysokiego limitu drastycznie wydłuża czas obliczeń, nie przekładając się na zauważalną zmianę zachowania algorytmu w większości standardowych sytuacji. Jedyną korzyścią z głębokiej analizy jest w tym wypadku zdolność do obrony przed złożonymi pułapkami, których przygotowanie wymaga od przeciwnika kilku tur. Z drugiej strony, skrajne zredukowanie tego parametru do głębokości równej 1 sprawia, że agent staje się całkowicie krótkowzroczny, a odniesienie zwycięstwa nie stanowi dla gracza ludzkiego najmniejszego wyzwania.

Rysunek 4: Wygrana człowieka na MiniMaxa o niskiej głębokości

```
INFO:root:Moving: P[x][HumanPlayer]
Select one of fields: [6, 7, 8, 9, 10, 11, 12] ? 12
INFO:root:Player selected 12
INFO:root:Move done.

Turn 10 moving: o, player P[o][MinMaxPlayer]
 ----
|ooox|
|oo  |
|    |
|xxxx|
 ----

RESULT:x:P[x][HumanPlayer]
Time played: {'o': 0.7343292236328125, 'x': 19014.52326774597}
```

Odpowiednio wysoka głębokość przeszukiwania pozwala algorytmowi zidentyfikować ścieżki prowadzące do nieuniknionej porażki i zablokować takie zagrożenie z odpowiednim wyprzedzeniem.

Rysunek 5: Obrona MinMaxa przed widłami

```
INFO:root:Moving: P[o][HumanPlayer]
Select one of fields: [5, 6, 7, 8, 9, 10, 11, 12, 13, 14] ? 14
INFO:root:Player selected 14
INFO:root:Move done.

Turn 7 moving: x, player P[x][MinMaxPlayer]
 ----
|oxxx|
|o   |
|    |
|  oo|
 ----

INFO:root:Moving: P[x][MinMaxPlayer]
INFO:root:Player selected 8
INFO:root:Move done.

Turn 8 moving: o, player P[o][HumanPlayer]
 ----
|oxxx|
|o   |
|x   |
|  oo|
 ----

INFO:root:Moving: P[o][HumanPlayer]
```

4 Algorytm Min-Max vs Algorytm Min-Max

4.1 Plansza 3x3

Starcie dwóch graczy wykorzystujących algorytm Min-Max pozbawione jest czynnika losowego. Ponieważ obaj gracze w identyczny sposób oceniają stany planszy i konsekwentnie wybierają pierwszy węzeł o najwyższej wartości użyteczności, rozgrywka jest w pełni deterministyczna. Każda kolejna partia rozegrana przy niezmienionych limitach głębokości przeszukiwań będzie idealną repliką poprzedniej, stanowiąc dowód na powtarzalność i ścisłą optymalizację decyzji podejmowanych przez algorytm.

Rysunek 6: MinMax z głębokością 9 vs MinMax z głębokością 6

```
INFO:root:Move done.

Turn 9 moving: x, player P[x][MinMaxPlayer]
 ---
|oox|
|xxo|
|oxo|
 ---

RESULT:DRAW
Time played: {'o': 102.60891914367676, 'x': 11.138916015625}
```

Analiza starć między dwoma agentami Min-Max ujawnia nieliniową zależność wyników od ustawionego limitu głębokości. Przy bardzo niskich wartościach (1–3) algorytmy są na tyle krótkowzroczne, że o zwycięstwie często decyduje przypadkowy układ znaków. Na poziomach 4 i 5 dochodzi do anomalii, w których jeden z graczy może wygrać dzięki zjawisku przesuwającego się horyzontu – po prostu w swojej turze jako pierwszy dostrzega koniec gry i wykorzystuje przewagę informacji. Pełna stabilizacja następuje dopiero od głębokości równej 6, gdzie perfekcyjna obrona z obu stron zawsze gwarantuje remis. Co niezwykle istotne, przewaga wynikająca z różnicy głębokości między graczami (np. starcie głębokości 2 z 5) ma praktyczne znaczenie wyłącznie na niższych poziomach (1–6), gdzie głębiej analizujący agent potrafi dostrzec pułapki i "o krok"wyprzedzić przeciwnika. Powyżej tego progu (np. w walce agenta o głębokości 7 z agentem o głębokości 9) asymetria parametrów traci jakiekolwiek znaczenie, ponieważ obaj gracze widzą już wystarczająco daleko, aby zniwelować każde zagrożenie i nieuchronnie doprowadzić do remisu.

Rysunek 7: MinMax z głębokością 5 vs MinMax z głębokością 5

```
INFO:root:Move done.

Turn 7 moving: x, player P[x][MinMaxPlayer]
 ---
|oxx|
|ooo|
|x  |
 ---

RESULT:o:P[o][MinMaxPlayer]
Time played: {'o': 9.54294204711914, 'x': 5.979061126708984}
```

4.2 Plansza 4x4

Podczas starcia dwóch algorytmów Min-Max na planszy 4x4 rozgrywka zawsze kończy się remisem, niezależnie od przyjętej głębokości przeszukiwania. Co ciekawe, przyczyna tego remisu jest zupełnie inna dla płytkiej i głębokiej analizy. Przy niskich wartościach tego parametru (np. 1–3) algorytmy dysponują zbyt wąskim horyzontem, aby zaplanować atak wymagający ułożenia aż czterech znaków. Zamiast budować złożone pułapki, jedynie zachowawczo zajmują wolne pola, a partia kończy się remisem z powodu ich niezdolności do stworzenia wygrywającej sekwencji. Z kolei przy wysokich wartościach głębokości, algorytmy zaczynają grać optymalnie i bezbłędnie blokują każdy ruch przeciwnika. Dowodzi to, że przy idealnej obronie obu stron wariant 4x4 jest grą nierozstrzygalną. Wniosek jest zatem jednoznaczny: w starciach symetrycznych na powiększonej planszy zwiększanie limitu głębokości nie zmienia ostatecznego, remisowego wyniku, a jedynie drastycznie wydłuża czas niezbędny na przeliczenie kolejnych ruchów.

5 Wnioski

Przeprowadzone symulacje potwierdzają, że kółko i krzyżyk to gra w pełni rozwiązana, która przy bezbłędnej obronie obu stron zawsze kończy się remisem. Wyniki wyraźnie pokazują, że zwiększanie głębokości przeszukiwania w algorytmie Min-Max nie poprawia jego skuteczności w sposób liniowy, lecz ujawnia tzw. efekt horyzontu – przy płytkiej analizie algorytm jest zbyt krótkowzroczny, by dostrzec i zablokować wieloetapowe pułapki, a na średnich

głębokościach wynik zależy od tego, który agent jako pierwszy przeanalizuje stany końcowe gry. Przejście na powiększoną planszę 4x4 ostatecznie obnaża ograniczenia wydajnościowe tej metody: ustawienie wysokich limitów przeszukiwania drastycznie wydłuża czas obliczeń, zupełnie nie zmieniając ostatecznego, remisowego rozstrzygnięcia partii. Udowadnia to, że klasyczny MinMax jest algorytmem deterministycznym i skrajnie asekuracyjnym, którego główną barierą w większych grach staje się złożoność obliczeniowa procesora.